ENGINEERING NOTEBOOK

Wireless Vehicle Telemetry & Control System

ESP32-Based Embedded Platform

Engineer	Start Date
Sai Palepu	March 2025

Personal Engineering Record

1.1 Project Summary

This notebook walks through the design, build, and testing of a wireless vehicle telemetry and control system built around an ESP32 microcontroller. The system is meant to act like a portable vehicle brain that can be mounted on a wheeled platform or eventually integrated into go-kart hardware. It reads physical sensor data in real time, transmits that data wirelessly to an external interface, and can accept command inputs to influence motor control — forming a closed-loop, networked embedded system.

This project was built to show practical experience with embedded systems programming, hardware interfacing, wireless communication, and controls integration — skill areas directly relevant to companies operating at the intersection of aerospace, high-performance vehicles, and advanced embedded platforms.

1.2 Engineering Goals

The project is structured around five core engineering goals, each of which builds on the last:

1. **Sensor Integration:** Wire and read data from an IMU (MPU-6050), a hall-effect speed sensor, and a temperature sensor (DS18B20). Convert raw I2C, GPIO interrupt, and 1-Wire readings into calibrated physical quantities.
2. **Firmware Architecture:** Write structured, interrupt-driven firmware in C++ using the Arduino framework on FreeRTOS. Handle sensor sampling, data filtering, PWM generation, and communication in concurrent tasks.
3. **Wireless Telemetry:** Transmit structured sensor packets over WiFi via MQTT at a controlled rate. Handle connection drops, reconnection logic, and variable latency gracefully.
4. **Real-Time Dashboard:** Build a browser-based dashboard to visualize live RPM, acceleration, attitude, temperature, motor duty, and system health. Data must update with less than 500 ms visible dashboard latency.
5. **Closed-Loop Control:** Implement a PI speed controller that uses hall-effect encoder feedback to drive a motor toward a commanded RPM setpoint, with the setpoint adjustable remotely over the wireless link.

1.3 System Architecture — Three-Layer Model

The system consists of three logical layers: the embedded hardware layer, the communication layer, and the client interface layer.

LAYER 1 — EMBEDDED HARDWARE (ESP32 + peripherals)

- Sensors: MPU-6050 IMU (I2C), hall-effect encoder (GPIO interrupt), DS18B20 temperature (1-Wire)
- Actuators: TB6612FNG / DRV8871 motor driver via PWM and direction control (LEDC peripheral)
- MCU: ESP32-WROOM-32D @ 240 MHz, dual-core, 4MB flash, WiFi + BT

LAYER 2 — COMMUNICATION (WiFi / MQTT)

- ESP32 connects as WiFi station to local access point (or acts as soft-AP)
- MQTT broker (Mosquitto) runs on laptop or Raspberry Pi on the same LAN
- Telemetry topic: wvtcs/telemetry | Command topic: wvtcs/command

LAYER 3 — CLIENT INTERFACE (Browser Dashboard)

- Node.js backend subscribes to MQTT, serves WebSocket to browser
- Browser renders live gauges and charts using Chart.js + HTML/CSS
- Command buttons publish setpoint changes back through the broker to the ESP32

1.4 Bill of Materials — Prototype Build

Component	Part / Model	Interface	Est. Cost	Source
Microcontroller	ESP32-WROOM-32D DevKit	—	$8	Amazon / AliExpress
IMU	MPU-6050 (GY-521 module)	I2C (SDA/SCL)	$3	Amazon
Speed Sensor	Hall-effect (A3144) + magnet	GPIO (interrupt)	$2	Amazon
Temp Sensor	DS18B20 waterproof probe	1-Wire + 4.7kΩ	$4	Amazon
Motor Driver	TB6612FNG / DRV8871, 12V-capable	PWM + direction	$6–$8	Amazon
Motor (test)	12V DC gear motor, 100 RPM	Motor driver	$12	Amazon
Chassis	2WD robot chassis kit	—	$15	Amazon
Power Supply	3S LiPo 1500mAh + BEC 5V reg	—	$20	Amazon
Logic level shifter	TXS0108E 8-channel	—	$4	Amazon
Resistors / caps	4.7kΩ, 10kΩ, 100nF decoupling	—	$3	Kit
Breadboard + wiring	830pt + jumper wires	—	$8	Amazon
TOTAL (est.)			~$84	

Note: a Raspberry Pi 4 or laptop running Mosquitto (MQTT broker) is required as part of the communication infrastructure but is not listed as a project-specific cost.

SECTION 2

ENTRY 01	System Requirements & Architecture Decisions	*Week 1, Day 1 — May 2025*

Objective

Before ordering hardware or writing code, I wanted to define what the system actually needed to do and make the major architecture choices early. This entry lays out the requirements and the first design decisions.

Functional Requirements

ID	Requirement	Priority	Verification
FR-01	System shall sample IMU at minimum 50 Hz	High	Logic analyzer / serial log
FR-02	System shall compute wheel RPM from encoder pulses	High	Compare to known RPM source
FR-03	System shall transmit telemetry packet every 100 ms (10 Hz)	High	MQTT timestamp diff
FR-04	System shall reconnect to MQTT broker within 5 s of drop	Medium	Unplug/replug test
FR-05	Dashboard shall display telemetry updates with < 500 ms visible latency	Medium	Stopwatch + visual check
FR-06	System shall accept remote speed setpoint over MQTT command topic	Medium	Send command, observe motor
FR-07	Motor speed shall track setpoint within ±10% at steady state	Medium	Tachometer measurement

ID	Requirement	Priority	Verification
FR-08	System shall log overtemp warnings to serial	Low	Inject simulated values

Non-Functional Requirements

- Power budget: ESP32, sensors, and logic electronics shall not exceed 2A from the 5V BEC. Motor current is supplied separately from the 12V battery rail through the motor driver.
- Firmware binary shall fit within 1MB of flash (leaving 3MB for OTA and data)
- WiFi credentials stored in NVS, not hardcoded in source
- All code version-controlled in Git from day one

Architecture Decision 1 — Communication Protocol

Decision: use MQTT over WebSockets rather than raw HTTP polling or UDP.

Options considered:

- HTTP GET/POST polling (client requests data every N ms)
- UDP broadcast (ESP32 sends, anything listens)
- MQTT via TCP (selected)
- Bluetooth SPP / BLE notifications

Rationale: MQTT is a publish-subscribe protocol designed for constrained IoT devices. It decouples the publisher (ESP32) from the subscriber (dashboard), which simplifies both sides. The broker handles message queuing if a client momentarily disconnects, and MQTT natively supports bidirectional messaging, needed for command-down / telemetry-up. HTTP polling would create unnecessary overhead and coupling; UDP would require custom reliability logic; BLE is limited in range and throughput.

The broker runs Mosquitto on a Raspberry Pi 4 or development laptop. In a future revision, the ESP32 could act as a soft-AP and host a lightweight WebSocket server, eliminating the external broker for standalone demo use.

Architecture Decision 2 — RTOS Task Structure

Decision: use FreeRTOS (built into ESP-IDF/Arduino) with three concurrent tasks pinned to specific cores.

Task	Core	Period	Responsibilities
sensorTask	Core 1	20 ms (50 Hz)	Read IMU via I2C, process hall-effect RPM timing, read temperature every 2 s
controlTask	Core 1	20 ms (50 Hz)	Run PI controller using latest filtered RPM estimate, compute PWM duty, write to LEDC
commTask	Core 0	100 ms (10 Hz)	Package JSON telemetry, publish to MQTT, receive commands

Core 0 handles the WiFi stack by default in ESP-IDF, so commTask runs there to avoid contention with sensor/control timing. Shared data between tasks is protected with a FreeRTOS mutex or atomic reads where appropriate.

Architecture Decision 3 — Sensor Package

IMU: MPU-6050 selected over alternatives (LSM6DS, BNO055) due to cost, ubiquity of documentation, and availability of the Wire library on ESP32. The DMP (Digital Motion Processor) on the MPU-6050 is initially bypassed; raw accel/gyro data is read and a complementary filter applied in firmware for attitude estimation.

Speed sensor: hall-effect pulse timing via GPIO interrupt is preferred over a quadrature encoder at this stage because the motor available has a single-channel magnet ring. An interrupt service routine (ISR) captures the time between rising edges. RPM is calculated from pulse period rather than from pulse count over a short 20 ms window, which is more accurate for a low-RPM, single-magnet wheel sensor.

Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
I2C bus conflicts / clock stretching	Med	High	Use 100 kHz I2C, add pull-ups, validate with logic analyzer
WiFi interference causing MQTT drops	Med	Med	Implement exponential backoff reconnect loop
Motor back-EMF corrupting sensor readings	Low	High	Decoupling caps on power rails, optoisolate if needed
Temperature sensor parasitic power failure	Low	Med	Use external power mode with 4.7kΩ pull-up
ESP32 watchdog reset under heavy WiFi load	Low	High	Feed watchdog in idle loop, check task stack sizes

Conclusions & Next Steps

The requirements are detailed enough to begin ordering parts and setting up the project. These architecture decisions give a baseline to compare later design choices against. The three-layer architecture (embedded → communication → interface) will guide module boundaries in code.

1. Order all components from BOM
2. Set up Git repository with folder structure: /firmware, /dashboard, /docs, /test
3. Install ESP-IDF + Arduino framework in VSCode with PlatformIO
4. Install Mosquitto MQTT broker on development laptop

ENTRY 02	Hardware Bring-Up: ESP32 & IMU (MPU-6050)	*Week 1, Day 3 — May 2025*

Objective

Verify that the ESP32 development board is functional, establish an I2C connection to the MPU-6050, and read raw accelerometer and gyroscope data with correct scaling. This is the first hardware milestone.

Hardware Setup

Components on bench: ESP32-WROOM-32D DevKit v1, GY-521 MPU-6050 module, 400pt breadboard, 3.3V from DevKit 3V3 pin.

ESP32 Pin	MPU-6050 Pin	Notes
3V3	VCC	3.3V supply — GY-521 has onboard 3.3V regulator, 5V input also OK
GND	GND	Common ground
GPIO 21 (SDA)	SDA	4.7kΩ pull-up to 3.3V on SDA
GPIO 22 (SCL)	SCL	4.7kΩ pull-up to 3.3V on SCL
GPIO 13 (optional)	INT	Interrupt pin — not used in initial test, kept for later

Firmware: I2C Scan & Register Read

Wrote a minimal I2C scanner first to confirm the MPU-6050 appears at address 0x68 (AD0 pin tied low), then wrote direct register reads to WHO_AM_I (0x75), which should return 0x68 as a device identifier.

```
Wire.begin(21, 22); // SDA, SCL
Wire.beginTransmission(0x68);
```

```
uint8_t err = Wire.endTransmission();
if (err == 0) Serial.println("MPU-6050 found at 0x68");
else Serial.printf("Error: %d\n", err);

// Wake device from sleep (register 0x6B = PWR_MGMT_1, write 0x00)
Wire.beginTransmission(0x68);
Wire.write(0x6B);
Wire.write(0x00);
Wire.endTransmission();
```

Results

WHO_AM_I register correctly returned 0x68. Device woke from sleep successfully. Raw accelerometer reads on all three axes with the gravity vector showing ~16384 LSB on the Z axis (device flat), matching the ±2g full-scale default (sensitivity = 16384 LSB/g).

Axis	Raw (LSB)	Converted (g)	Expected
AX	-42	-0.003 g	~0 (flat)
AY	+35	+0.002 g	~0 (flat)
AZ	+16351	+0.999 g	~1.0 (gravity down)

Gyroscope reads are ~0 ±2 LSB while stationary, consistent with expected noise floor. Full-scale default is ±250°/s (sensitivity = 131 LSB/°/s).

Issues Encountered

Issue 1: initial reads returned garbage. Root cause: forgot to call Wire.begin() before any I2C transaction, and did not wake the device from sleep mode. The MPU-6050 powers up in sleep mode by default — PWR_MGMT_1 register bit 6 must be cleared.

Issue 2: occasional I2C timeout on rapid successive reads. Root cause: no delay between endTransmission() and the next beginTransmission(). Added a 2 ms delay as a workaround; will investigate DMA I2C in the ESP-IDF native API if timing becomes critical.

Conclusions & Next Steps

IMU bring-up successful. I2C communication confirmed stable at 100 kHz. Next step is to implement a proper MPU6050 class that encapsulates register reads, applies calibration offsets, and outputs calibrated floats in physical units (m/s², deg/s).

5. Write MPU6050 driver class with calibration routine
6. Implement complementary filter for roll/pitch estimation
7. Wire hall-effect sensor and verify interrupt-based pulse counting

ENTRY 03	IMU Driver, Calibration & Complementary Filter	*Week 2, Day 1 — May 2025*

Objective

Build a complete MPU-6050 software driver, establish a calibration procedure to remove constant bias from gyroscope readings, and implement a complementary filter to fuse accelerometer and gyroscope data into stable roll and pitch estimates.

Calibration Procedure

Gyroscopes have a constant DC bias (offset) that causes the integrated angle to drift even when the device is stationary. To measure this, I hold the sensor still for 500 samples and average the readings; these offsets are subtracted from every subsequent measurement.

```
// CALIBRATION ALGORITHM
Place sensor perfectly flat and stationary for 5 seconds.
Sample all 6 axes (AX, AY, AZ, GX, GY, GZ) 500 times with 10ms delay.
Compute mean of each axis.
Accel offsets: AX_off = mean(AX), AY_off = mean(AY),
               AZ_off = mean(AZ) - 16384
               (subtract 1g from Z since gravity is real, not an error)
Gyro offsets:  GX_off = mean(GX), GY_off = mean(GY), GZ_off = mean(GZ)
Store offsets in global struct; apply before every read.
```

Measured Calibration Offsets

Channel	Raw Mean (LSB)	Offset Applied	Post-Cal @ Rest
AX	-42	-42	0 ± 2 LSB
AY	+35	+35	0 ± 2 LSB
AZ	+16351	-33 (16384-16351)	16384 ± 3 LSB
GX	+18	+18	0 ± 1 LSB
GY	-12	-12	0 ± 1 LSB
GZ	+7	+7	0 ± 1 LSB

Complementary Filter

A complementary filter fuses high-frequency data from the gyroscope (which drifts slowly) with low-frequency data from the accelerometer (which is noisy but drift-free) using a weighted sum. The filter constant α controls the tradeoff.

```
// Accelerometer-derived angles (absolute, noisy):
roll_accel  = atan2(AY_cal, AZ_cal) * 180/PI
pitch_accel = atan2(-AX_cal, sqrt(AY_cal^2 + AZ_cal^2)) * 180/PI

// Gyroscope integration (relative, drifts):
roll_gyro  += (GX_cal / 131.0) * dt   // 131 LSB per deg/s
pitch_gyro += (GY_cal / 131.0) * dt

// Complementary fusion (alpha = 0.98):
roll  = alpha * (roll  + gyro_dt) + (1 - alpha) * roll_accel
pitch = alpha * (pitch + gyro_dt) + (1 - alpha) * pitch_accel
```

Alpha = 0.98 was selected after empirical testing. Higher alpha trusts the gyro more (smoother but drifts); lower alpha trusts the accelerometer more (noisier but stable long-term). At 0.98 with 50 Hz sampling, the filter settles within ~1 second after a disturbance.

Validation Test

Slowly rotated the sensor through a known 90° rotation (using a protractor as reference). Filter output settled to 89.4° ± 0.6° — acceptable for this application. IMU temperature compensation was not implemented; will revisit if gyro drift proves problematic in extended runs.

Conclusions & Next Steps

IMU driver with calibration and complementary filter is functional. Roll and pitch estimates are stable to within ±1° at rest and track manual rotations accurately. Next: wire the speed sensor and implement RPM measurement.

8. Wire hall-effect sensor (A3144) to GPIO with 10kΩ pull-up
9. Implement ISR-based period measurement, compute RPM in controlTask
10. Wire DS18B20 temperature sensor on 1-Wire bus

ENTRY 04	Speed Sensor (Hall-Effect Encoder) & RPM Calculation	*Week 2, Day 3 — May 2025*

Objective

Wire the A3144 hall-effect sensor to detect magnetic pulses from a rotating wheel. Implement an interrupt service routine (ISR) to measure the time between pulses and compute RPM from pulse period — a more accurate approach than counting pulses in a short sampling window for a single-magnet, low-RPM sensor.

Hardware Wiring

A3144 Pin	ESP32 Pin	Notes
VCC	3V3	3.3V supply
GND	GND	Common ground
OUT	GPIO 34 + 10kΩ to 3V3	GPIO34 is input-only, ideal for ISR

A small neodymium magnet (4mm disc) was epoxied to the wheel hub at one location; each full revolution produces one pulse. For higher resolution, a multi-pole magnet ring can be used — noted as a future enhancement.

ISR Implementation

```
volatile uint32_t lastPulseTime_us = 0;
volatile uint32_t pulseInterval_us = 0;
portMUX_TYPE mux = portMUX_INITIALIZER_UNLOCKED;
const uint8_t PULSES_PER_REV = 1;

void IRAM_ATTR hallISR() {
  uint32_t now = micros();
  portENTER_CRITICAL_ISR(&mux);
  if (lastPulseTime_us > 0) {
    pulseInterval_us = now - lastPulseTime_us;
  }
  lastPulseTime_us = now;
  portEXIT_CRITICAL_ISR(&mux);
}

// Called from controlTask at 50 Hz.
// RPM is calculated from pulse period, not pulses counted in a 20 ms window.
float computeRPM() {
  uint32_t intervalCopy, lastPulseCopy;
  portENTER_CRITICAL(&mux);
  intervalCopy = pulseInterval_us;
  lastPulseCopy = lastPulseTime_us;
  portEXIT_CRITICAL(&mux);

  // If no pulse has arrived recently, assume the wheel has stopped.
  if (micros() - lastPulseCopy > 1500000UL) return 0.0f;
  if (intervalCopy == 0) return 0.0f;
  return 60000000.0f / (intervalCopy * PULSES_PER_REV);
}
```

The ISR is marked IRAM_ATTR to ensure it runs from IRAM (not flash), preventing cache-miss delays during interrupt execution — critical for accurate timing on the ESP32.

RPM Validation

Spun the motor at known voltages and compared computed RPM to the motor's rated no-load speed (100 RPM @ 12V). Applied 12V, measured 97 RPM — within 3%, acceptable given slight load from the encoder magnet and timing variation in the single-pulse measurement.

Because the wheel uses one magnet, pulse-counting over a 20 ms window is too noisy at low speed. Instead, RPM is calculated from the measured time between pulses. A stale timeout of 1.5 seconds is used so the system reports 0 RPM only when no pulse has arrived for longer than the expected interval at very low speed.

Conclusions & Next Steps

Speed sensor is functional. RPM computed with sufficient accuracy for the PI controller. Next steps: temperature sensor wiring, then consolidate all sensor reads into the sensorTask FreeRTOS structure.

11. Wire DS18B20, verify 1-Wire communication
12. Integrate all sensors into sensorTask running at 50 Hz
13. Implement shared data struct protected by mutex

ENTRY 05	Temperature Sensor, FreeRTOS Task Structure & Shared State	*Week 2, Day 5 — May 2025*

Objective

Wire the DS18B20 temperature sensor and integrate all sensor reads into a structured FreeRTOS firmware architecture with proper shared-state management between tasks.

DS18B20 Wiring & Read

DS18B20 Pin	ESP32 Pin	Notes
VDD	3V3	External power mode (not parasitic)
GND	GND	Common ground
DATA	GPIO 4 + 4.7kΩ to 3V3	1-Wire bus, strong pull-up required

Using the OneWire + DallasTemperature Arduino libraries. DS18B20 conversion time is 750 ms at 12-bit resolution. Since temperature changes slowly, the sensor is polled every 2 seconds using a timer check inside sensorTask rather than blocking the task for 750 ms.

FreeRTOS Task Architecture

```
// Shared state structure (protected by mutex)
struct TelemetryData {
  float accel_x, accel_y, accel_z;  // m/s^2
  float gyro_x, gyro_y, gyro_z;     // deg/s
  float roll, pitch;                // degrees (complementary filter)
  float rpm;                        // wheel RPM
  float temperature_c;              // C
  float motor_duty;                 // 0.0 - 1.0
  float speed_setpoint;             // RPM target
  uint32_t timestamp_ms;            // millis()
} telemetry;

SemaphoreHandle_t dataMutex;

// Task creation in setup():
dataMutex = xSemaphoreCreateMutex();
xTaskCreatePinnedToCore(sensorTask,  "sensor",  4096, NULL, 2, NULL, 1);
```

```
xTaskCreatePinnedToCore(controlTask, "control", 4096, NULL, 2, NULL, 1);
xTaskCreatePinnedToCore(commTask,    "comm",    8192, NULL, 1, NULL, 0);
```

Stack size for commTask is 8KB due to JSON serialization and WiFi stack usage. Priority 2 for sensor and control tasks ensures real-time sensor reads are not preempted by communication overhead.

Mutex-Protected Data Access Pattern

```
// In sensorTask: writing data
if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(5)) == pdTRUE) {
  telemetry.roll = computed_roll;
  telemetry.rpm = computed_rpm;
  telemetry.timestamp_ms = millis();
  xSemaphoreGive(dataMutex);
}

// In commTask: reading for transmission
TelemetryData snapshot;
if (xSemaphoreTake(dataMutex, pdMS_TO_TICKS(10)) == pdTRUE) {
  snapshot = telemetry;   // copy entire struct atomically
  xSemaphoreGive(dataMutex);
}
// serialize 'snapshot' — no need to hold the mutex during serialization
```

Conclusions & Next Steps

All three sensors are integrated and running concurrently in a structured FreeRTOS architecture. The shared data mutex prevents race conditions between tasks. Serial monitor shows stable 50 Hz sensor output with no task watchdog resets after 30 minutes of continuous operation.

14. Implement WiFi connection management and MQTT client
15. Define JSON telemetry packet format
16. Test MQTT publish from ESP32 to Mosquitto broker on laptop

SECTION 3

ENTRY 06	WiFi Connection Management & MQTT Client	*Week 3, Day 1 — May 2025*

Objective

Implement robust WiFi connection handling and integrate an MQTT client library to publish telemetry from the ESP32 to a Mosquitto broker running on the development laptop, handling reconnection automatically.

WiFi Architecture Decision

WiFi credentials (SSID and password) are stored in NVS (Non-Volatile Storage) rather than hardcoded. For initial development, a #define in a config.h file excluded from Git (via .gitignore) is used and manually provisioned to NVS on first flash — mirroring how production devices handle credential management.

Connection State Machine

```
States: DISCONNECTED -> WIFI_CONNECTING -> WIFI_CONNECTED
        -> MQTT_CONNECTING -> MQTT_CONNECTED

Transitions:
  DISCONNECTED    -> WIFI_CONNECTING : always; WiFi.begin() called
```

```
  WIFI_CONNECTING -> WIFI_CONNECTED  : WiFi.status() == WL_CONNECTED
  WIFI_CONNECTING -> DISCONNECTED    : timeout > 10s; increment retry count
  WIFI_CONNECTED  -> MQTT_CONNECTING : immediately
  MQTT_CONNECTING -> MQTT_CONNECTED  : client.connect() returns true
  MQTT_CONNECTING -> DISCONNECTED    : timeout > 5s
  MQTT_CONNECTED  -> DISCONNECTED    : client.connected() returns false

Backoff: retry_delay = min(30s, 1s * 2^retryCount)   // exponential backoff
```

MQTT Telemetry Packet Format

Telemetry is serialized as a JSON object and published to the topic wvtcs/telemetry. JSON was chosen over binary encoding (e.g. MessagePack, protobuf) for initial development due to ease of debugging; a binary format can replace it if bandwidth becomes a concern.

```
{
  "ts": 1716890423150,
  "roll": -2.34, "pitch": 0.87,
  "ax": -0.03, "ay": 0.02, "az": 9.81,
  "gx": 0.1, "gy": -0.2, "gz": 0.0,
  "rpm": 94.3, "temp": 28.5,
  "duty": 0.47, "sp": 100.0
}
// Packet size: ~220 bytes. At 10 Hz = 2.2 kB/s — well within WiFi capacity.
```

MQTT Subscription — Command Topic

The ESP32 also subscribes to wvtcs/command. Command messages are JSON with a single field specifying the new speed setpoint; the callback parses the payload and updates the shared telemetry struct's speed_setpoint field through the mutex.

```
// Published by dashboard to wvtcs/command:
{ "setpoint": 75.0 }   // target RPM

// ESP32 callback:
void mqttCallback(char* topic, byte* payload, unsigned int length) {
  // parse JSON, extract setpoint
  // acquire mutex, update telemetry.speed_setpoint, release mutex
}
```

Test Results

Connected the ESP32 to a laptop hotspot with Mosquitto running on default config, port 1883. Used MQTT Explorer (desktop GUI) to observe incoming telemetry — packets arrived consistently at 9–11 Hz (target: 10 Hz) and JSON parsed correctly. Published a command packet manually and observed the ESP32 serial log updating the setpoint field.

Conclusions & Next Steps

WiFi and MQTT communication layer is operational; telemetry is flowing from sensors through firmware to broker. Next: build the motor driver integration and PI controller so there is meaningful control data to include in telemetry.

17. Wire 12V-capable motor driver, test manual PWM control
18. Implement PI speed controller in controlTask
19. Tune P and I gains on bench

SECTION 4

ENTRY 07	Motor Driver Wiring & PWM Bring-Up	*Week 3, Day 3 — May 2025*

Objective

Wire a 12V-capable DC motor driver, configure the ESP32 LEDC PWM peripheral, and verify manual speed control of the DC gear motor before adding closed-loop logic.

Motor Driver Wiring

Motor Driver Pin	Connected To	Notes
VM / VIN	12V battery	Motor supply for 12V gear motor
GND	Common GND	Shared with ESP32 GND
PWM	GPIO 25 (PWM)	Motor speed command from ESP32 LEDC peripheral
DIR / IN1	GPIO 26	Direction control, held forward for this test
AOUT1	Motor +	Motor terminal A
AOUT2	Motor –	Motor terminal B
nSLEEP / EN	3V3	Pull high to enable driver
nFAULT	GPIO 27 + 10kΩ to 3V3	Fault detection input (optional)

LEDC PWM Configuration

```
// LEDC = LED Control peripheral, used for all general PWM on ESP32
const int PWM_CHANNEL = 0;
const int PWM_FREQ_HZ = 20000;   // 20 kHz: above human hearing
const int PWM_RESOLUTION = 10;   // 10-bit = 0-1023 range
const int PWM_PIN = 25;

ledcSetup(PWM_CHANNEL, PWM_FREQ_HZ, PWM_RESOLUTION);
ledcAttachPin(PWM_PIN, PWM_CHANNEL);

// Set duty cycle (0 = off, 1023 = full on):
void setMotorDuty(float duty_0_to_1) {
  uint32_t duty = (uint32_t)(duty_0_to_1 * 1023.0f);
  ledcWrite(PWM_CHANNEL, duty);
}
```

20 kHz was chosen because it is above the audible range, reducing motor whine, and is a common PWM frequency for small DC motor driver testing. The 10-bit resolution provides 1024 discrete duty-cycle steps — sufficient for smooth speed control at this scale.

Manual PWM Test Results

Duty Cycle	Measured RPM	Motor Current (mA)	Notes
0%	0	0	Motor off
25%	18	95	Below minimum spin threshold for this motor
30%	31	108	Minimum reliable spin
50%	54	135	Linear region

Duty Cycle	Measured RPM	Motor Current (mA)	Notes
75%	79	162	Linear region
100%	97	188	Near rated no-load speed

The motor has a deadband below ~28% duty cycle where it does not spin. The PI controller output must account for this by clamping minimum duty to 0.28 when a nonzero setpoint is commanded.

Conclusions & Next Steps

Motor driver functional. PWM linearization noted: deadband at low duty is a known characteristic of brushed DC motors that must be handled in the controller. Next: implement the PI controller.

20. Implement PI controller in controlTask
21. Test step response with setpoint changes via MQTT command
22. Tune gains, measure overshoot and settling time

ENTRY 08	PI Speed Controller Design & Tuning	*Week 3, Day 5 — May 2025*

Objective

Implement a proportional-integral (PI) speed controller that drives motor RPM toward a commanded setpoint using real-time encoder feedback, and tune gains to achieve stable, accurate speed tracking.

PI Controller Theory

A PI controller computes an output (motor duty cycle) based on the error between the desired setpoint and the measured process variable (RPM). The proportional term responds instantly to current error; the integral term accumulates past error to eliminate steady-state offset.

```
error(t) = setpoint - measured_rpm
P_term = Kp * error(t)
I_term += Ki * error(t) * dt
output = P_term + I_term
output = clamp(output, 0.28, 1.0)   // deadband floor + saturation ceiling

// Anti-windup: if output is saturated (at 0.28 or 1.0),
// do NOT accumulate I_term in that direction.
// (prevents integrator windup during startup or hard limits)
```

Gain Tuning Process

Tuned gains empirically using the Ziegler–Nichols heuristic as a starting point, then refined by observation. Target: setpoint tracking within ±10% RPM, settling time < 3 seconds, no sustained oscillation.

Iter.	Kp	Ki	Behavior Observed
1	0.005	0.0	P-only: steady-state error of ~15 RPM at 100 RPM setpoint
2	0.005	0.001	Integral too slow: 8s to eliminate offset
3	0.005	0.003	Good tracking, slight overshoot on step (~12%)
4	0.004	0.003	Reduced overshoot (~6%), settling ~2.2s — SELECTED
5	0.003	0.004	Sluggish response, acceptable but worse than #4

Step Response Test — Final Gains (Kp = 0.004, Ki = 0.003)

Commanded a step from 0 RPM to 100 RPM via MQTT command. RPM was estimated using pulse-period measurement from the hall-effect sensor and lightly filtered before being used by the PI controller. Bench results:

- Rise time (10% to 90% of setpoint): 1.4 seconds
- Overshoot: 6.1% (106 RPM peak)
- Settling time (within ±5 RPM): 2.2 seconds
- Steady-state error: approximately 1–2 RPM after filtering

Performance meets FR-07 (within ±10% at steady state). The overshoot is acceptable for this application. In a future revision, a rate limiter on the setpoint input (ramp instead of step) would reduce overshoot without needing to retune.

Conclusions & Next Steps

PI controller is tuned and functional. The system now forms a complete closed loop: the hall sensor estimates RPM from pulse timing, the controller computes duty, the motor responds, and the updated RPM estimate feeds back into the next control cycle — the core embedded control demonstration. Next: build the dashboard interface.

23. Build Node.js MQTT subscriber backend
24. Build browser dashboard with live charts
25. Add command UI to send setpoint changes from browser

SECTION 5

ENTRY 09	Node.js Backend & WebSocket Bridge	*Week 4, Day 1 — May 2025*

Objective

Build a lightweight Node.js server that subscribes to the MQTT telemetry topic and relays data to browser clients via WebSocket, bridging the MQTT protocol (used by the ESP32) with the WebSocket protocol (native to browsers).

Architecture

The backend uses three Node.js packages: mqtt (MQTT client), ws (WebSocket server), and express (static file server for the HTML dashboard). The server runs on port 3000 (HTTP) and 8080 (WebSocket).

```
const mqtt = require('mqtt');
const WebSocket = require('ws');
const mqttClient = mqtt.connect('mqtt://localhost:1883');
const wss = new WebSocket.Server({ port: 8080 });

mqttClient.on('connect', () => {
  mqttClient.subscribe('wvtcs/telemetry');
  mqttClient.subscribe('wvtcs/command'); // echo commands back for logging
});

mqttClient.on('message', (topic, message) => {
  // Broadcast incoming telemetry to ALL connected browser clients
  wss.clients.forEach(client => {
    if (client.readyState === WebSocket.OPEN) client.send(message.toString());
  });
});

// Browser -> ESP32 command relay:
```

```
wss.on('connection', ws => {
  ws.on('message', msg => mqttClient.publish('wvtcs/command', msg));
});
```

Dashboard Latency Measurement

For latency testing, the Node.js bridge added a receive timestamp when each MQTT telemetry packet arrived. The browser compared this bridge timestamp to its own receipt time, giving a practical measurement of MQTT-to-browser dashboard delay. Across 500 packets, dashboard-side delay remained below 300 ms, satisfying FR-05 (< 500 ms visible update latency). A future revision should add a true round-trip test where the browser sends a timestamped command and the ESP32 echoes it back.

Conclusions & Next Steps

Backend bridge operational; all telemetry reaching the browser in real time. Next: build the visual dashboard HTML/JS interface with gauges and charts.

26. Design HTML dashboard layout with RPM gauge, roll/pitch display, and temperature
27. Implement live Chart.js rolling time-series for RPM and duty cycle
28. Add setpoint slider and send button wired to WebSocket

ENTRY 10	Browser Dashboard: Design & Implementation	*Week 4, Day 3 — May 2025*

Objective

Build an HTML/CSS/JavaScript browser dashboard that visualizes all incoming telemetry in real time and allows the operator to command speed setpoints back to the ESP32.

Dashboard Layout Design

The dashboard is divided into four panels: a status bar (connection state, packet rate, timestamp), a gauges row (RPM, temperature, motor duty), a time-series chart (rolling 30-second window of RPM vs. setpoint), and an attitude display (roll and pitch as numeric values with visual bars).

The control panel includes a numeric input for speed setpoint (0–100 RPM) and a Send button that publishes the command over WebSocket to the Node.js bridge, which relays it to MQTT, which delivers it to the ESP32.

Key JavaScript — WebSocket & Chart Update

```
const ws = new WebSocket('ws://localhost:8080');
const rpmData = []; // Rolling buffer, max 300 points (30s at 10Hz)

ws.onmessage = (event) => {
  const d = JSON.parse(event.data);
  updateGauge('rpm-gauge', d.rpm, 0, 120);
  updateGauge('temp-gauge', d.temp, 20, 80);
  document.getElementById('roll').textContent  = d.roll.toFixed(1)  + '°';
  document.getElementById('pitch').textContent = d.pitch.toFixed(1) + '°';
  rpmData.push({ x: Date.now(), y: d.rpm });
  if (rpmData.length > 300) rpmData.shift();
  chart.update('none'); // Chart.js no-animation update for performance
};

// Send setpoint command:
document.getElementById('send-btn').onclick = () => {
  const sp = parseFloat(document.getElementById('sp-input').value);
  ws.send(JSON.stringify({ setpoint: sp }));
};
```

Demonstrated Capability — Full System Test

With all components running simultaneously, performed the following demonstration sequence:

29. Powered on ESP32. Observed WiFi and MQTT connection within 3.2 seconds.
30. Dashboard loaded in Chrome. WebSocket connected. Packet counter incrementing at 10 Hz.
31. Set setpoint to 60 RPM via dashboard. Observed motor spin up, RPM chart tracking to ~60.
32. Physically tilted ESP32 board 45°. Roll gauge on dashboard updated in real time.
33. Raised setpoint to 100 RPM. Observed PI controller step response on chart: small overshoot and settling in roughly 2–3 seconds after RPM filtering.
34. Disconnected laptop from WiFi access point for 8 seconds. ESP32 reconnected automatically within 4.7 seconds. Data resumed.

All functional requirements FR-01 through FR-08 were exercised during this test session, with formal verification summarized in Section 6.

Entry 10 Conclusions

The full system is operational. The dashboard successfully closes the loop from physical sensor to human interface and back to motor control. At this stage the project demonstrates embedded firmware, real-time sensor processing, wireless communication, and web-based visualization working as an integrated system.

SECTION 6

ENTRY 11	Formal Requirements Verification	*Week 4, Day 5 — May 2025*

Verification Summary

Each functional requirement was formally verified against the acceptance criteria defined in Entry 01. Results are documented below.

ID	Requirement	Test Method	Result	Status
FR-01	IMU sampled at ≥50 Hz	Serial timestamp diff over 1000 samples	Mean: 19.8ms (50.5 Hz)	**Pass**
FR-02	RPM computed from encoder	Compare to stroboscope reference	±3% at 100 RPM after filtering	**Pass**
FR-03	Telemetry at 10 Hz	MQTT Explorer timestamp analysis	9.8–10.2 Hz observed	**Pass**
FR-04	MQTT reconnect within 5s	Unplug router, re-plug, measure time	Max observed: 4.7s	**Pass**
FR-05	Dashboard latency <500ms	Node.js bridge timestamp comparison, 500 packets	Delay below 300ms	**Pass**
FR-06	Remote setpoint command	Publish command from dashboard, observe motor	Setpoint updated within 120ms	**Pass**
FR-07	Speed tracking ±10%	Steady-state RPM at 3 setpoints	~1–2 RPM steady-state error after filtering	**Pass**
FR-08	Overtemp warning logged	Heat DS18B20 with soldering iron tip	Warning logged at 55°C threshold	**Pass**

All eight functional requirements were verified at the bench test level. The system satisfies its original design intent for a prototype embedded telemetry and control platform.

Stress Testing

- Continuous operation for 2 hours: no crashes, no watchdog resets, no memory leak detected (heap free stable at 142 KB)
- 10 simulated WiFi disconnect/reconnect cycles: all reconnected successfully within the 5-second requirement
- Setpoint sweep from 30 to 100 RPM in 5 RPM steps: steady-state tracking error generally remained within 1–2 RPM after filtering
- Simultaneous IMU motion and speed change: no task priority inversion or mutex deadlock observed

Known Limitations & Future Work

Limitation	Impact	Planned Resolution
Single magnet: 1 pulse/rev limits update rate	Med: RPM estimate responds slowly at low speed	Multi-pole magnet ring or quadrature encoder
No OTA (over-the-air) firmware update	Low: requires USB to update	Implement OTA partition in flash layout
JSON telemetry larger than needed	Low: adequate for current use	Replace with MessagePack binary encoding
No SSL/TLS on MQTT	Med: security gap on open networks	Add TLS + certificate auth for production
Complementary filter: no magnetometer	Low: yaw drifts	Add HMC5883L magnetometer for full 9-DOF
Motor deadband not auto-calibrated	Low: fixed 28% floor	Add startup deadband characterization routine

SECTION 7

7.1 What This Project Demonstrates

This project was not simply about making an RC car or a sensor display. The deliberate goal was to build a system that mirrors the architecture of real embedded platforms used in aerospace, automotive, and robotics, and to understand each layer of that architecture from first principles.

SKILLS DEMONSTRATED

- Embedded C++ firmware: interrupt-driven ISRs, FreeRTOS task management, mutex-protected shared state, LEDC PWM peripheral, I2C/1-Wire/GPIO hardware drivers, NVS credential storage
- Sensor integration: IMU bring-up and calibration, complementary filter design, pulse-period RPM measurement, temperature sensing with conversion timing management
- Control systems: PI controller design, gain tuning methodology, anti-windup implementation, step response characterization (rise time, overshoot, settling time)
- Wireless networking: MQTT publish/subscribe, connection state machine with exponential backoff, WiFi station mode, bidirectional command/telemetry architecture
- Software systems: JSON serialization, WebSocket bridge server (Node.js), browser-based real-time data visualization, dashboard latency measurement
- Systems engineering: formal requirements definition, ADR documentation, BOM creation, requirements verification matrix, risk identification and mitigation

7.2 Key Engineering Decisions Revisited

Looking back at the three Architecture Decision Records from Entry 01:

Architecture Decision 1, MQTT: Validated

MQTT proved the right choice. The decoupled pub/sub model made it easy to add the dashboard without modifying the ESP32 firmware at all — the broker handled all routing. The bidirectional command channel worked without any additional architectural changes.

Architecture Decision 2, FreeRTOS tasks: Validated with one correction

The three-task structure worked well. One lesson: the initial stack allocation for commTask (4KB) was too small — the MQTT library's internal buffers plus JSON serialization caused a stack overflow on long topic strings. Increasing to 8KB resolved this. Always allocate headroom in RTOS stacks.

Architecture Decision 3, Sensor selection: Validated

MPU-6050 was the right choice for this stage. The complementary filter produced more than adequate attitude estimates. The hall-effect sensor's single-pulse-per-rev limitation is real but manageable using pulse-period RPM measurement and a stale-timeout stop condition. A quadrature encoder would be the first hardware upgrade.

7.3 Connection to Broader Engineering Context

The architecture of this system is structurally identical to systems used in professional contexts that motivated this project:

1. Vehicle/aircraft telemetry: sensors stream data at fixed rates, are packaged into structured messages, transmitted over a communication bus or RF link, and displayed on ground-station dashboards. The physics is different, but the information-flow architecture is similar.
2. Drone flight controllers: IMU data is fused with complementary or Kalman filters, motor outputs are controlled through PWM, and telemetry is sent to a ground-control station. This project is a simplified version of that same sensor-control-telemetry loop.
3. Industrial SCADA systems: sensors send data to controllers, which publish over MQTT or OPC-UA to a SCADA dashboard. This project is a miniaturized functional analog of that architecture.

7.4 What I Would Do Differently

If restarting this project, I would make three changes from the beginning. First, I would use the ESP-IDF native API instead of the Arduino framework — the Arduino layer adds convenience but hides important details about WiFi task scheduling and I2C DMA that are worth understanding directly. Second, I would design the PCB from day one rather than using a breadboard — most of the debugging effort was related to breadboard contact reliability and floating grounds, not the actual logic. Third, I would instrument the firmware with timing measurements from the start using ESP32 hardware timers to produce accurate profiling data rather than estimating from serial logs.

7.5 Next Phase: Planned Enhancements

- OTA firmware update support (ESP-IDF OTA API, update from dashboard)
- Kalman filter replacing complementary filter for IMU fusion
- CAN bus interface (add MCP2515 module) — directly relevant to vehicle ECU communication
- Integration with go-kart chassis: mount on kart, add GPS (NEO-M8N) for speed/position telemetry
- Battery voltage monitoring via ADC with low-voltage warning
- SD card data logging (SPI) for offline post-analysis

END OF ENGINEERING NOTEBOOK — WVTCS-001

All entries reviewed and signed by the engineer of record.

Engineer Signature	Date
Sai Palepu	05/02/2026